
Software Design Document

SmartSchool — Student Records, Fee Invoicing & Staff Payroll Management System

Prepared for: **ICT University Yaoundé**

Course: **SEN4121 – Large System Environment** (Practical Examination, Summer 2026)

Examiner: Engr. Tanwi Nkiamboh

Document version: 1.0
Group number: Group 2
Date: August 17, 2026
Status: Living document – updated through Week 4

Group Member	Matricule
Kwete Rayan	ICTU20241377
Kamdeu Yamdjeuson Neil Marshall	ICTU20241386
Younga Tchappi Dylane	ICTU2241836
Fru Chi Ehud Neba	ICTU20241812

This document describes the architecture, use cases, data model, and behavioural design of SmartSchool, a microservice-based school ERP built for the SEN4121 practical examination series. It reflects the system as actually implemented through Week 4 (environment/auth, database, microservices/gateway, messaging/security/performance), and marks Week 5/6 elements explicitly as planned.

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions and Abbreviations	2
1.4	Document Overview	2
1.5	Group Contributions	3
2	System Architecture	4
2.1	Architectural Style	4
2.2	Component Diagram	4
2.3	Component Responsibilities	4
3	Use Case Model	6
3.1	Actors	6
3.2	Use Case Diagram	6
3.3	Key Use Case Specification: Enroll in a Course	7
3.4	Key Use Case Specification: Record a Grade	7
4	Static Structure – Data Model	8
4.1	Overview	8
4.2	Notable Design Decisions	8
4.3	Normalization (3NF)	9
4.4	UML Class Diagram	9
5	Dynamic Behaviour	10
5.1	Sequence Diagram: Registration and Login	10
5.2	Sequence Diagram: Enroll in a Course (Async Invoice Creation)	10
5.3	Sequence Diagram: Record a Grade	12
5.4	State Transition Diagram: Invoice	12
5.5	Activity Diagram: Enrollment-to-Invoice Workflow	12
5.6	Object Diagram: Sample Enrollment Snapshot	12
5.7	Idempotent Event Consumption	13
6	API Design	16
6.1	Routing Table (Gateway)	16
6.2	Event Contract: <code>enrollment.created</code>	16
7	Security Design	18
7.1	Authentication and Session Model	18
7.2	Defense in Depth	18
7.3	OWASP Top 10 Mitigations	18
8	Performance and Reliability	19

8.1	Rate Limiting	19
8.2	Load Testing	19
8.3	Fault Tolerance in the Async Workflow	19
9	Deployment View	20
9.1	Container Topology	20
10	Planned Work (Weeks 5–6)	21

Chapter 1

Introduction

1.1 Purpose

This Software Design Document (SDD) describes the design of SmartSchool, a school ERP built as a set of independently deployable microservices behind a single API Gateway. It captures the system's architecture, static structure (data model), and dynamic behaviour, and is intended to guide implementation, the weekly oral evaluations, and future maintenance.

1.2 Scope

SmartSchool is organised into three functional modules, mirroring the exam brief's Academic / Finance / HR split:

- **Academic** — students, courses, enrollments, and grades.
- **Finance** — invoices, generated automatically from enrollments.
- **HR / Payroll** — staff records and payroll (data model complete; service layer planned for Week 5).

Cross-cutting concerns – authentication, the API Gateway, asynchronous messaging, and security – are treated as their own architectural layer rather than duplicated per module.

1.3 Definitions and Abbreviations

See the SRS (Section 1.3) for the full glossary; the same terms apply here.

1.4 Document Overview

Chapter 2 gives the system architecture (component diagram). Chapter 3 presents the use-case model (use case diagram). Chapter 4 presents the static structure (ER diagram and UML class diagram). Chapter 5 presents dynamic behaviour (three sequence diagrams, a state diagram, an activity diagram, and an object diagram). Chapter 6 documents the API design. Chapter 7 covers security. Chapter 8 covers performance and reliability. Chapter 9 covers deployment. Chapter 10 lists planned work for Weeks 5–6. All eight UML diagrams (every one except the ER diagram, which uses ER notation rather than UML) are generated from PlantUML source files under `documentation/figures/smartschool/`, so the diagram and the text describing it can never silently drift apart – regenerating a figure is a one-line command, not a manual redraw.

1.5 Notation Used in This Document

Every diagram in this document uses standard UML notation, drawn at full strength (activation bars, guard conditions, composition/aggregation, `<<include>>`/`<<extend>>`, swimlanes) rather than a simplified subset. Table ?? is a single reference point for every symbol used, so each diagram's

own section can focus on what it shows about SmartSchool specifically, not on re-explaining UML itself.

Table 1.1: UML and ER notation used throughout this document.

Symbol	Meaning	Appears in
Stick-figure actor	A human role (primary) interacting with the system	Use case diagram
Actor icon outside the system box, to the right	A <i>secondary</i> actor: a supporting, non-human system the use case depends on, never the initiator	Use case diagram
Oval	A use case: one discrete unit of system behaviour	Use case diagram
Dashed arrow, <<include>>	Base use case <i>always</i> incorporates the target use case's behaviour (arrow points to the included one)	Use case diagram
Dashed arrow, <<extend>>	The <i>source</i> use case optionally adds behaviour to the target base use case, under a condition (arrow points to the base)	Use case diagram
Box, 3 compartments	A class: name, attributes, operations	Class diagram
Italic name/operation	Abstract class or abstract operation – cannot be instantiated / has no body	Class diagram
<<interface>> stereotype	A contract with operations but no attributes or implementation	Class diagram
Hollow triangle, solid line	Inheritance / generalization (“is-a”)	Class diagram
Hollow triangle, dashed line	Realization – a class implements an interface	Class diagram
Filled diamond	Composition – the part's lifecycle is bound to the whole	Class, object diagrams
Hollow diamond	Aggregation – a weaker “has-a”; the part can outlive or be reassigned from the whole	Class, object diagrams
Plain line + multiplicity	Association, with how many instances relate on each end	Class, object diagrams
Underlined name : Class	An object diagram node: one specific instance, not a type	Object diagram
Box with plug icon	A component: a replaceable, encapsulated unit of implementation	Component diagram
Circle (“lollipop”)	A provided interface – what a component offers	Component diagram
Dashed arrow, open head	A required interface or a dependency – what a component needs	Component diagram
Vertical dashed line	A lifeline: one participant's timeline, top to bottom	Sequence diagrams
Solid arrow, filled head	A synchronous call – the caller waits for a reply	Sequence diagrams
Solid arrow, open head	An asynchronous message – the sender does not wait	Sequence diagrams

Table 1.1: UML and ER notation used throughout this document.

Symbol	Meaning	Appears in
Dashed arrow	A return / reply message	Sequence diagrams
Narrow rectangle on a lifeline	Activation bar: the participant is actively processing for that span	Sequence diagrams
alt / else frame	Alternative fragments – mutually exclusive branches	Sequence diagrams
Rounded rectangle	An action (one step)	Activity diagram
Diamond	A decision (one incoming, multiple guarded outgoing) or merge (multiple incoming, one outgoing)	Activity diagram
Solid horizontal bar	Fork (splits into concurrent flows) or join (synchronizes them back)	Activity diagram
Vertical column with a header	A swimlane – which participant owns each action beneath it	Activity diagram
Filled circle / bordered filled circle	Initial node / activity-final node	Activity diagram
Rounded rectangle (state diagram)	A state, optionally with entry/exit/do actions	State diagram
event [guard] / effect	On a transition: what triggers it, the condition that must hold, and the side effect it causes	State diagram

1.6 Group Contributions

Group Member	Contribution
Kwete Rayan	
Kamdeu Yamdjeuson Neil Marshall	
Younga Tchappi Dylane	
Fru Chi Ehud Neba	

Chapter 2

System Architecture

2.1 Architectural Style

SmartSchool follows a **microservice architecture behind an API Gateway**: every external client (the React frontend, curl, Postman, the examiner) talks only to the Gateway, which is the sole component with a port published to the host. Backend services are reachable only from other containers on the internal Docker network. This satisfies the Week 3 build task directly and gives each module (Academic, Finance) an independent deployment and scaling unit.

Design principle. The Finance Service never receives a direct call to create an invoice. It reacts to an `enrollment.created` event on its own schedule, so the Academic Service's request path is never coupled to Finance Service's availability or latency.

2.2 Component Diagram

Figure 2.1 is a standard UML component diagram (notation key in Table ??) rather than a generic box-and-arrow sketch:

- **Package boundary** – the five components that make up SmartSchool itself (React Frontend, API Gateway, Auth Service, Academic Service, Finance Service) are grouped inside one package, visually separating “the system” from the external Browser actor and the two infrastructure nodes (MySQL, RabbitMQ).
- **Provided interface (lollipop)** – Gateway API `/api/v1/**` is the *only* interface exposed outside the package; the Frontend depends on it with a dashed arrow (a *required* interface / dependency), and nothing outside the package can reach Auth Service, Academic Service, or Finance Service directly – there is deliberately no lollipop on any of them.
- **Dependency arrows (dashed, open arrowhead)** – every “requires” relationship: the Gateway on all three backend services (`requireAuth`, `proxy`), each service on MySQL (`mysql2 pool`), and the asynchronous publish/deliver pair through RabbitMQ.
- **Database and queue nodes** – MySQL and RabbitMQ use their dedicated UML node shapes rather than generic components, since they are infrastructure the components depend on, not components of SmartSchool itself.

2.3 Component Responsibilities

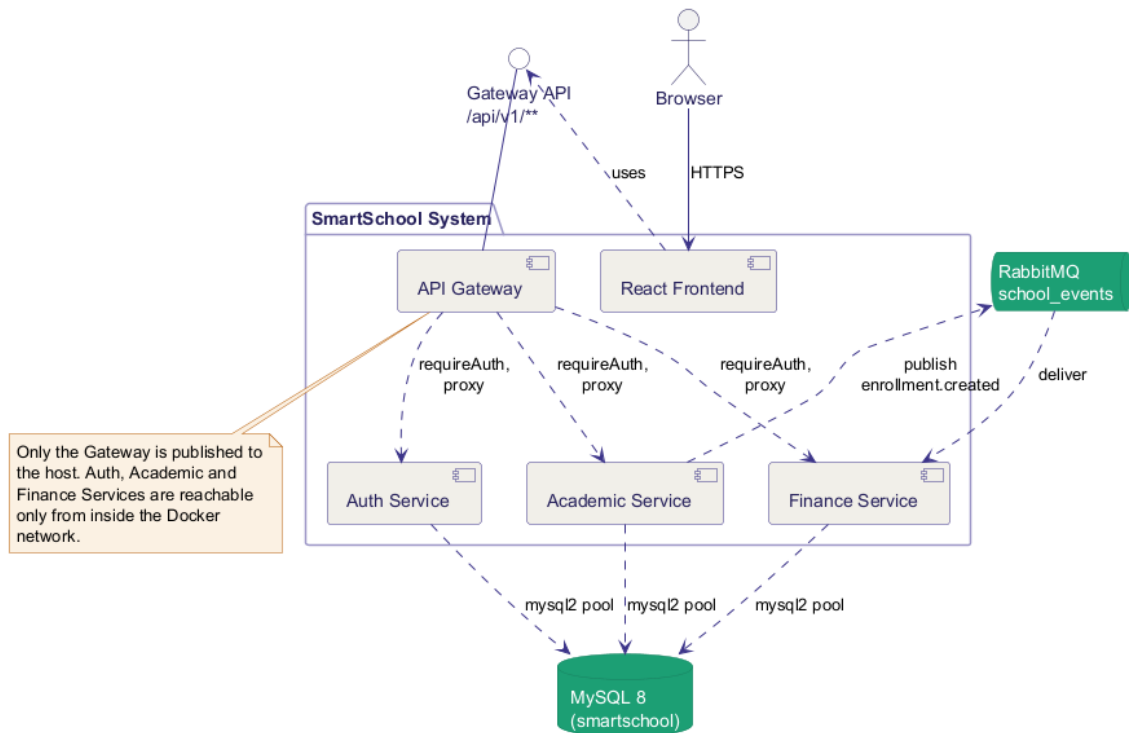


Figure 2.1: UML component diagram for SmartSchool: the Gateway is the only component with a provided interface reachable from outside the system package; every other dependency is internal (PlantUML source: figures/smartschool/09_component.puml)

Component	Responsibility
API Gateway	Single public entry point. Verifies JWT signature/expiry on protected routes <i>before</i> proxying (<code>requireAuth</code>); routes by URL prefix via <code>http-proxy-middleware</code> ; rate-limits every route except <code>/health</code> ; serves a combined Swagger UI at <code>/docs</code> . Does <i>not</i> enforce role/ownership rules – that is left to the owning service.
Auth Service	Owns the <code>users</code> table. Handles registration (<code>bcrypt</code> hashing), login (JWT issuance), <code>/me</code> , and a demo reporting endpoint gated to <code>admin/teacher</code> .
Academic Service	Owns <code>students</code> , <code>courses</code> , <code>enrollments</code> , <code>grades</code> . Enforces role/ownership rules per endpoint (e.g. a teacher may only grade their own courses). Publishes <code>enrollment.created</code> on successful enrollment.
Finance Service	Owns <code>invoices</code> . Serves <code>GET /api/invoices</code> and independently consumes <code>enrollment.created</code> to create invoices.
MySQL 8	Single shared relational database (<code>smartschool</code>), one schema across all three modules, accessed by each service through its own connection pool with parameterized queries only.
RabbitMQ 3.13	Message broker; one durable direct exchange (<code>school_events</code>) and one durable queue (<code>invoice_creation_queue</code>) bound with routing key <code>enrollment.created</code> . Management UI on <code>:15672</code> for live inspection.

Component	Responsibility
React Frontend	Single-page application (Vite + React). Holds the JWT and user profile in context, persisted to <code>localStorage</code> ; calls the Gateway exclusively, never a backend service directly.

Chapter 3

Use Case Model

3.1 Actors

Actor	Kind	Description
Admin	Primary	Creates courses, enrolls students on their behalf, views reports; highest privilege.
Teacher	Primary	Records grades for courses they teach; views reports alongside Admin.
Student	Primary	Registers, logs in, enrolls in courses, views their own enrollments, grades, and invoices.
Message Broker (RabbitMQ)	Secondary	Non-human, supporting actor: carries the <code>enrollment.created</code> event that drives asynchronous invoice creation. Never initiates a use case itself; supports one.

3.2 Use Case Diagram

Figure 3.1 shows the three primary (human) actors, the secondary (system) actor positioned on the right per UML convention, and the full use case set with explicit `<<include>>` and `<<extend>>` relationships:

- `<<include>>` (arrow from base use case to included use case): every functional use case includes *Log In* – it is a mandatory precondition, factored out once rather than repeated on each use case.
- `<<extend>>` (arrow from extending use case to base use case): *Create Invoice (async)* extends *Enroll in a Course* – it is optional, conditional behaviour (only fires once the event is consumed) rather than a step the base use case always performs itself.

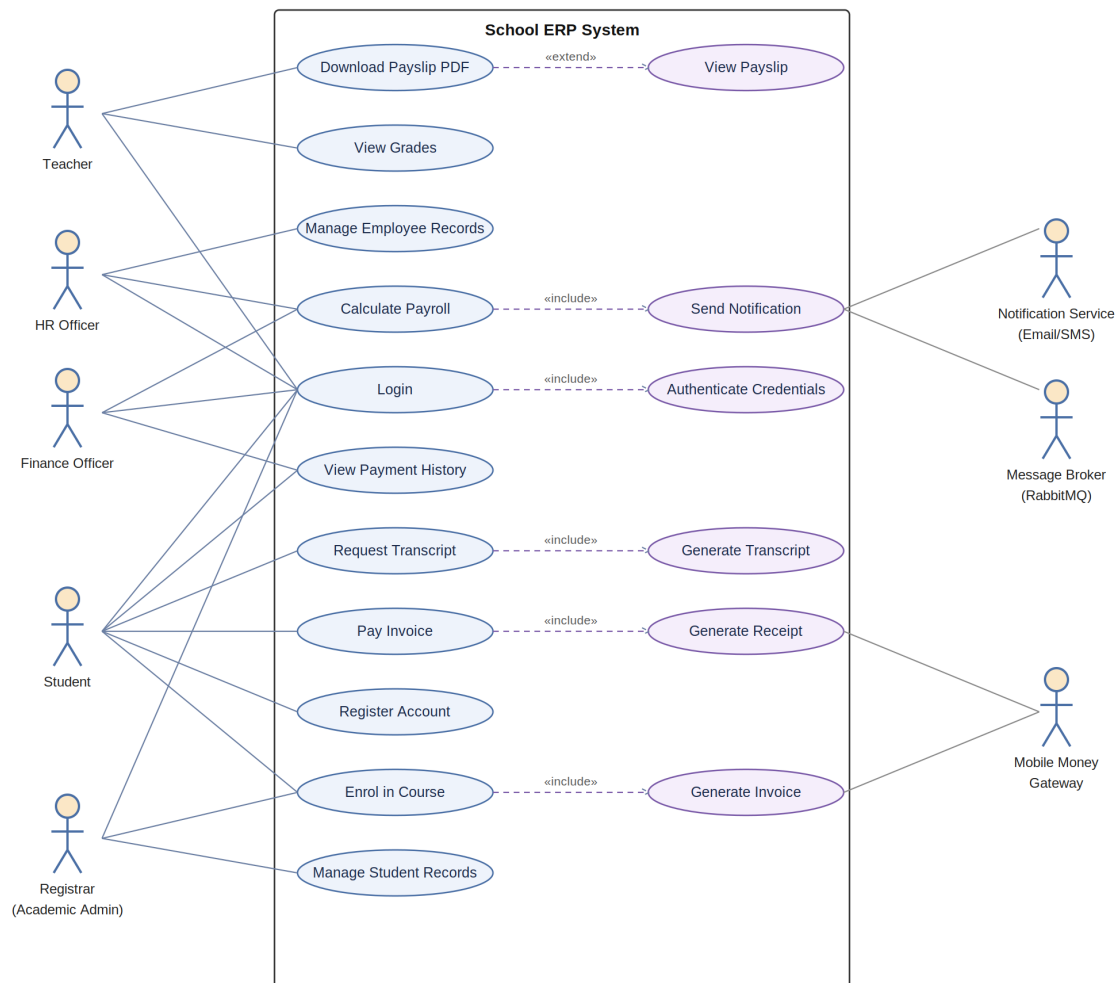


Figure 3.1: Use case diagram for SmartSchool: primary actors, the Message Broker as a secondary actor, and `<<include>>`/`<<extend>>` relationships (PlantUML source: figures/smartschool/01_usecase.puml)

3.3 Key Use Case Specification: Enroll in a Course

Field	Description
Primary actor	Student (or Admin, on a student's behalf)
Preconditions	Actor is authenticated; the target course exists
Main flow	(1) Actor calls <code>POST /api/v1/enrollments</code> via the Gateway; (2) Gateway verifies the JWT and forwards to Academic Service; (3) Academic Service inserts the enrollment and commits it; (4) Academic Service publishes <code>enrollment.created</code> and returns 201 immediately; (5) Finance Service consumes the event and creates a pending invoice, independently
Alternate flow	Duplicate enrollment → 409, no event published. Broker/Finance Service temporarily down → enrollment still succeeds; the event stays durably queued and is processed once the consumer reconnects
Postconditions	Enrollment recorded; an invoice for it eventually appears without further action from the actor

3.4 Key Use Case Specification: Record a Grade

Field	Description
Primary actor	Teacher (or Admin)
Preconditions	Actor is authenticated with role <code>admin</code> or <code>teacher</code> ; the enrollment exists
Main flow	(1) Actor calls <code>POST /api/v1/grades</code> with enrollment id, assessment type, and score; (2) Academic Service verifies the JWT; (3) if the caller is a Teacher, the service checks the enrollment's course is taught by that teacher; (4) the grade is recorded
Alternate flow	A Teacher attempting to grade a course they do not teach receives 403
Postconditions	The grade is recorded and visible to the student, the recording teacher, and any Admin

Chapter 4

Static Structure – Data Model

4.1 Overview

Figure 4.1 shows the eight tables of the `smartschool` schema and their relationships, spanning the shared identity table and the three functional modules. The full DDL lives in `database/schema.sql`.

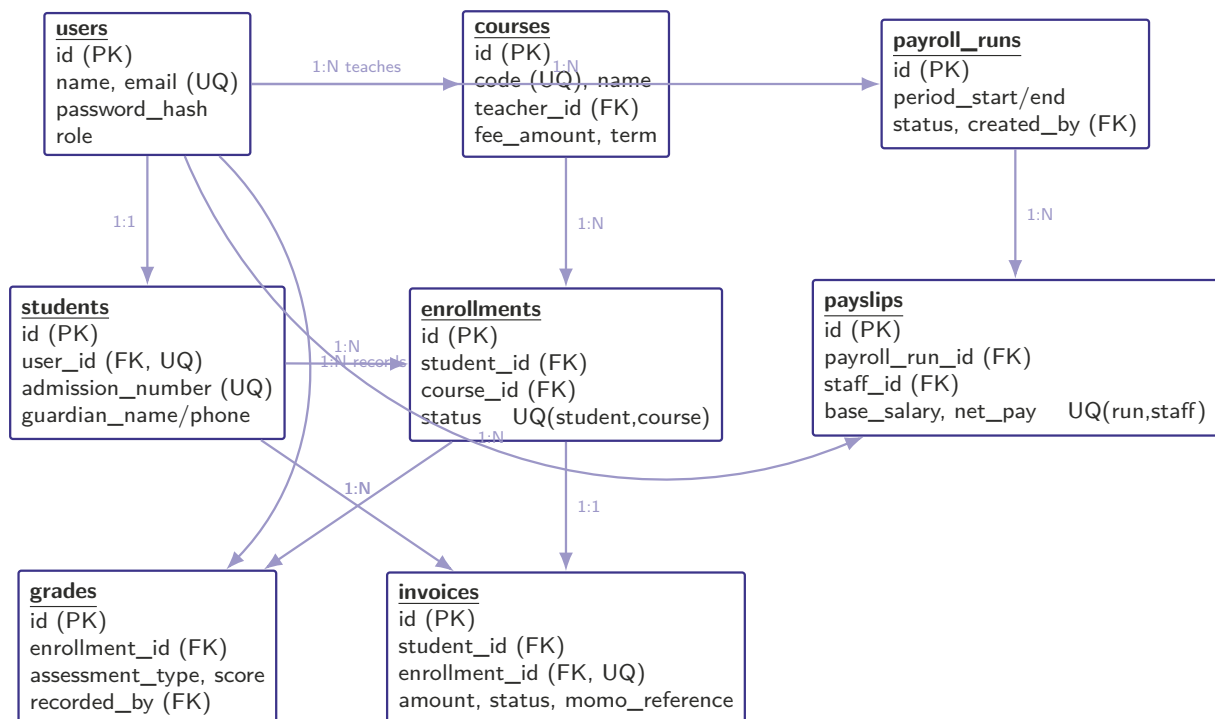


Figure 4.1: Entity-Relationship Diagram for SmartSchool (PK = primary key, FK = foreign key, UQ = unique constraint). Payroll tables (right) are implemented in the schema but not yet exposed by a service.

4.2 Notable Design Decisions

- `students` is a separate table, 1:1 with `users`, rather than adding academic-only columns to `users` – keeps authentication identity (login/role) separate from domain profile data.
- There is no separate `teachers` table: a teacher is simply a `users` row with `role = 'teacher'`; `courses.teacher_id` and `grades.recorded_by` reference `users.id` directly.
- `invoices.amount` is deliberately a *price snapshot at enrollment time*, not a live reference to `courses.fee_amount` – this is not a 3NF violation, since it represents a historical fact (“this enrollment was invoiced for this amount”), and a later fee change must not retroactively alter an issued invoice.
- `invoices.enrollment_id` is `UNIQUE`, enforcing at the database level that an enrollment can never

produce more than one invoice – this is what makes the event consumer’s duplicate-delivery handling (Section 5.7) safe without extra application-level locking.

- Money columns use `DECIMAL(10,2)`, never `FLOAT/REAL`, to avoid floating-point rounding error on currency.

4.3 Normalization (3NF)

Every table is in Third Normal Form: every non-key column depends on the whole primary key and nothing else. Fixed value sets (`role`, `status`, `assessment_type`, `method`) use native `ENUM` types rather than free text, preventing invalid values at the schema level. Indexes are added on every foreign key expected to be filtered on in a live query (`courses.teacher_id`, `enrollments.student_id/course_id`, `grades.enrollment_id/recorded_by`, `invoices.student_id`, `payslips.staff_id`), beyond the exam’s 2-index minimum.

4.4 UML Class Diagram

Where Figure 4.1 above is a *database-oriented* ER diagram (tables, columns, keys), Figure 4.2 is the corresponding *UML class diagram*: the same domain model expressed as classes with typed attributes, key operations, enumerations, and every relationship kind the UML metamodel distinguishes:

- **Abstraction** – `Person` is an abstract class (italicized, with an abstract operation); `Payable` is an interface realized by two unrelated classes.
- **Inheritance** (hollow triangle) – `Admin`, `Teacher`, and `Student` all specialize `Person`.
- **Realization** (dashed hollow triangle) – `Invoice` and `Payslip` both realize `Payable`.
- **Composition** (filled diamond) – `Student` *-- `Enrollment`, `Enrollment` *-- `Grade`, `Enrollment` *-- `Invoice`: the part cannot outlive the whole.
- **Aggregation** (hollow diamond) – `Course` o-- `Enrollment`, `PayrollRun` o-- `Payslip`: a weaker “has-a” where the part’s lifecycle is not strictly bound to the whole.
- **Plain association** with multiplicities – e.g. `Teacher` -- `Course`, `Admin` -- `PayrollRun`.

This is a *logical/OOP* view; the physical schema (Figure 4.1) implements the `Person` hierarchy as a single `users` table with a `role` discriminator column (single-table inheritance) rather than one table per subclass, as noted on the diagram itself.

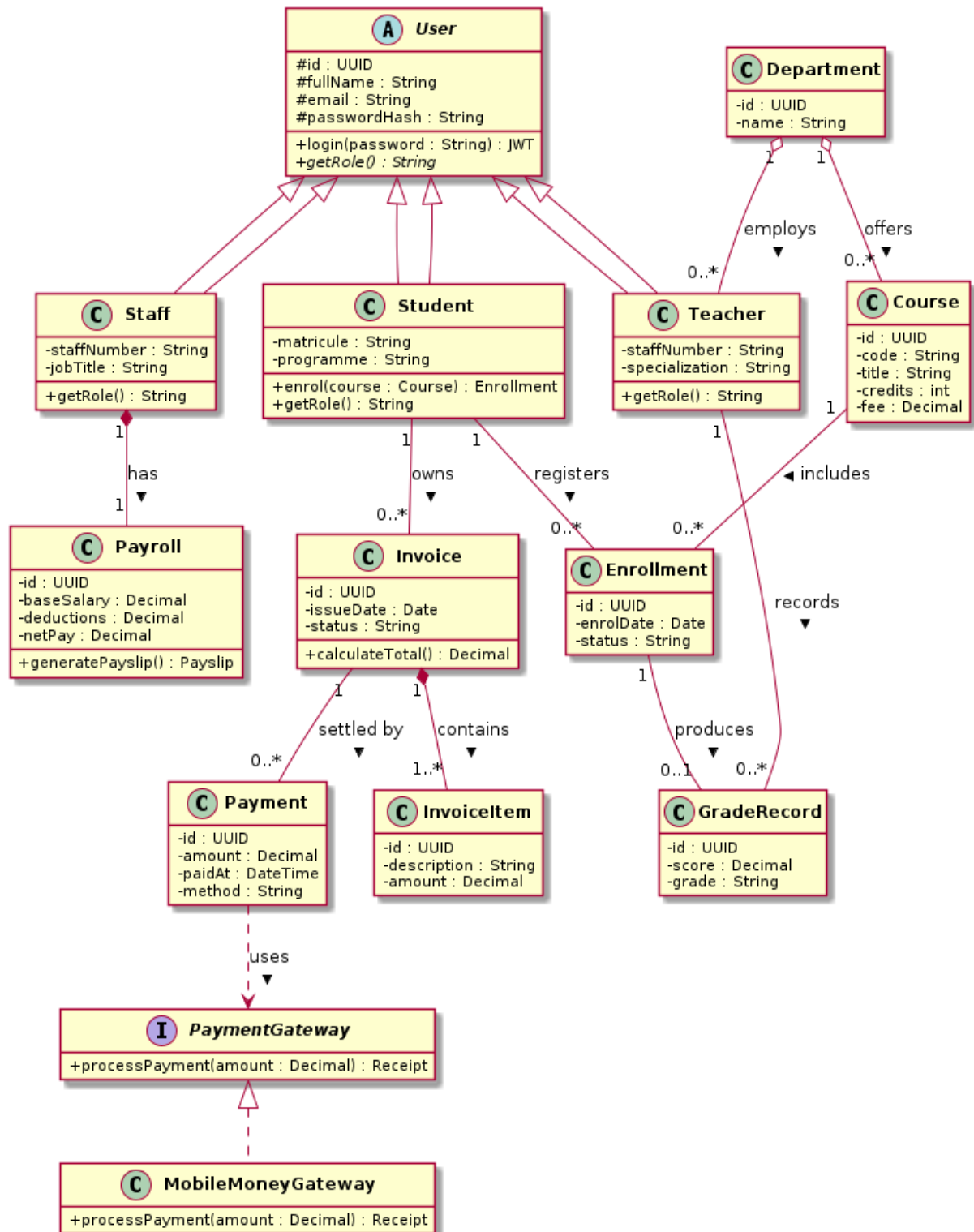


Figure 4.2: UML class diagram for SmartSchool, demonstrating abstraction, inheritance, realization, composition, and aggregation (PlantUML source: figures/smartschool/02_class.puml)

Chapter 5

Dynamic Behaviour

5.1 Sequence Diagram: Registration and Login

Figure 5.1 traces account creation and authentication. Every lifeline carries an explicit **activation bar**: it is drawn active only for the span between receiving a call and sending its reply, so nested calls (Gateway waiting on Auth Service, Auth Service waiting on MySQL) are visually unambiguous.

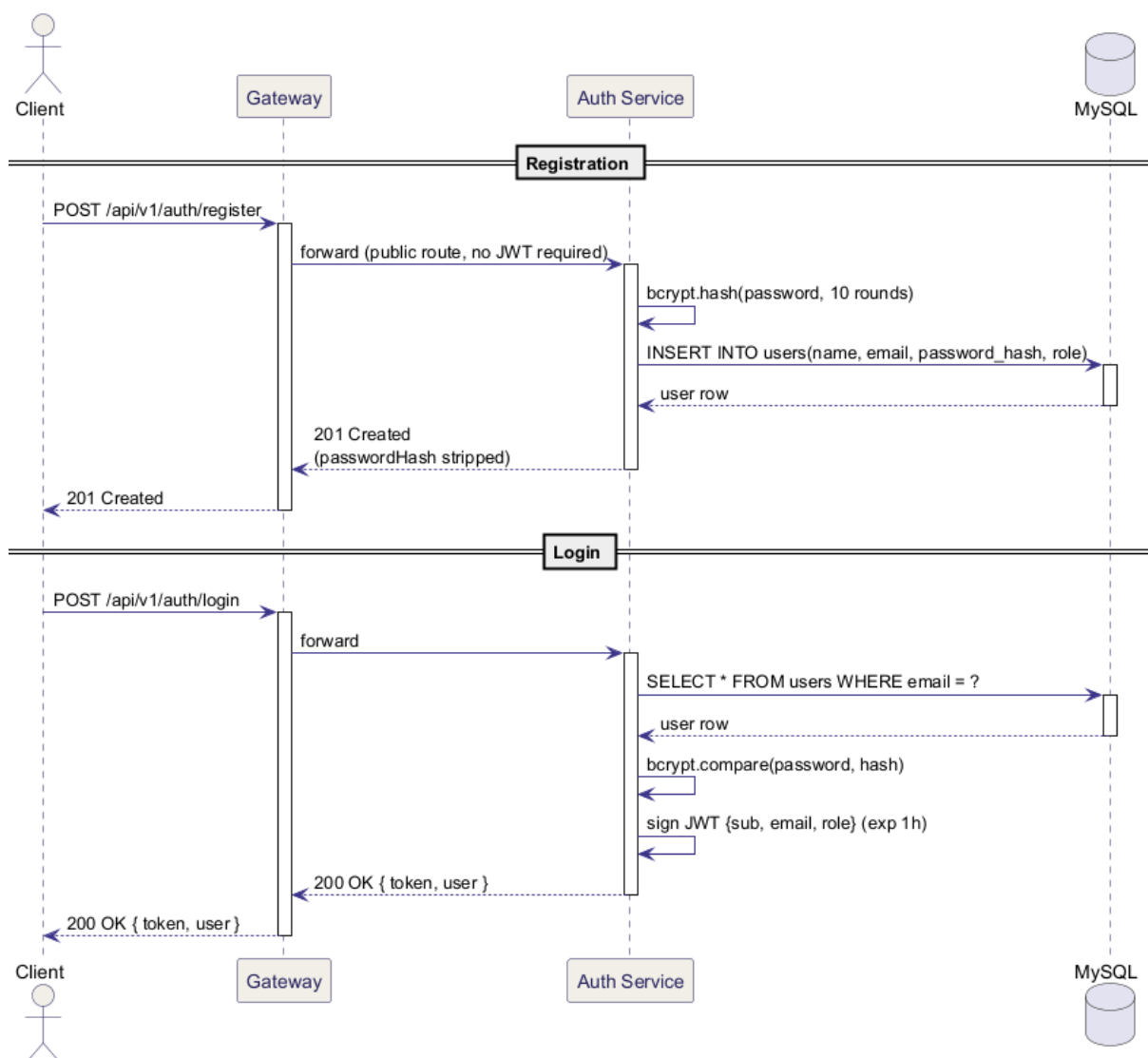


Figure 5.1: Sequence diagram: registration and login, with activation bars on every lifeline (PlantUML source: figures/smartschool/03_seq_auth.puml)

5.2 Sequence Diagram: Enroll in a Course (Async Invoice Creation)

Figure 5.2 shows the central asynchronous workflow: the Academic Service never waits for the Finance Service. Note where Academic Service's activation bar ends – *before* the “later, independently” separator – versus Finance Service's activation bar, which only opens once RabbitMQ delivers the message, well after the Student already has a response. The publish and deliver arrows use the open (async) arrowhead, not the filled synchronous one.

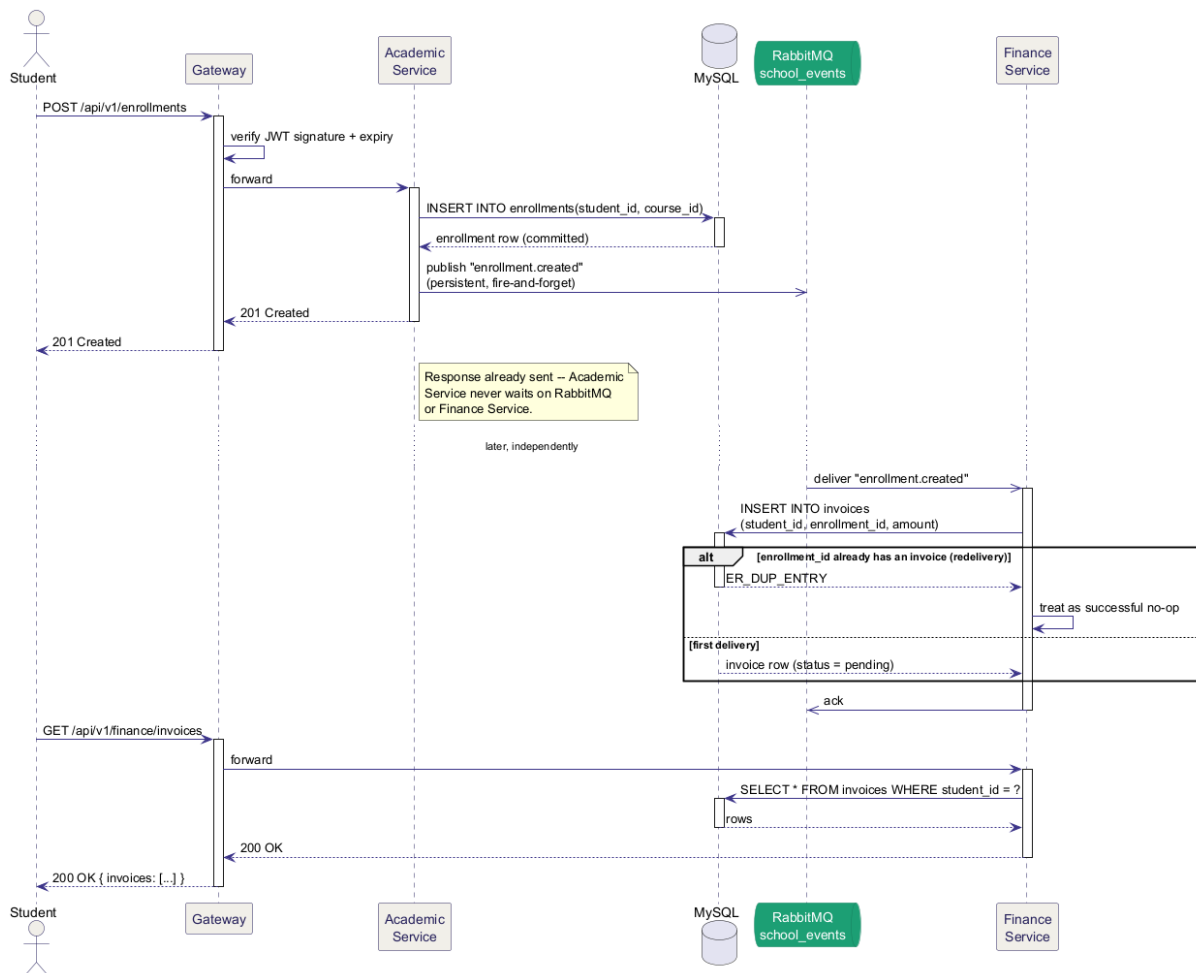


Figure 5.2: Sequence diagram: enrollment triggers asynchronous invoice creation, with activation bars showing exactly when each participant is doing work (PlantUML source: figures/smartschool/04_seq_enrollment_invoice.puml)

5.3 Sequence Diagram: Record a Grade

Figure 5.3 shows the ownership check that restricts a Teacher to grading only the courses they teach, enforced independently by Academic Service even though the Gateway already confirmed the caller is logged in. The `alt` fragment's two branches each close their own activation bars independently, which is standard UML for divergent outcomes of a single call.

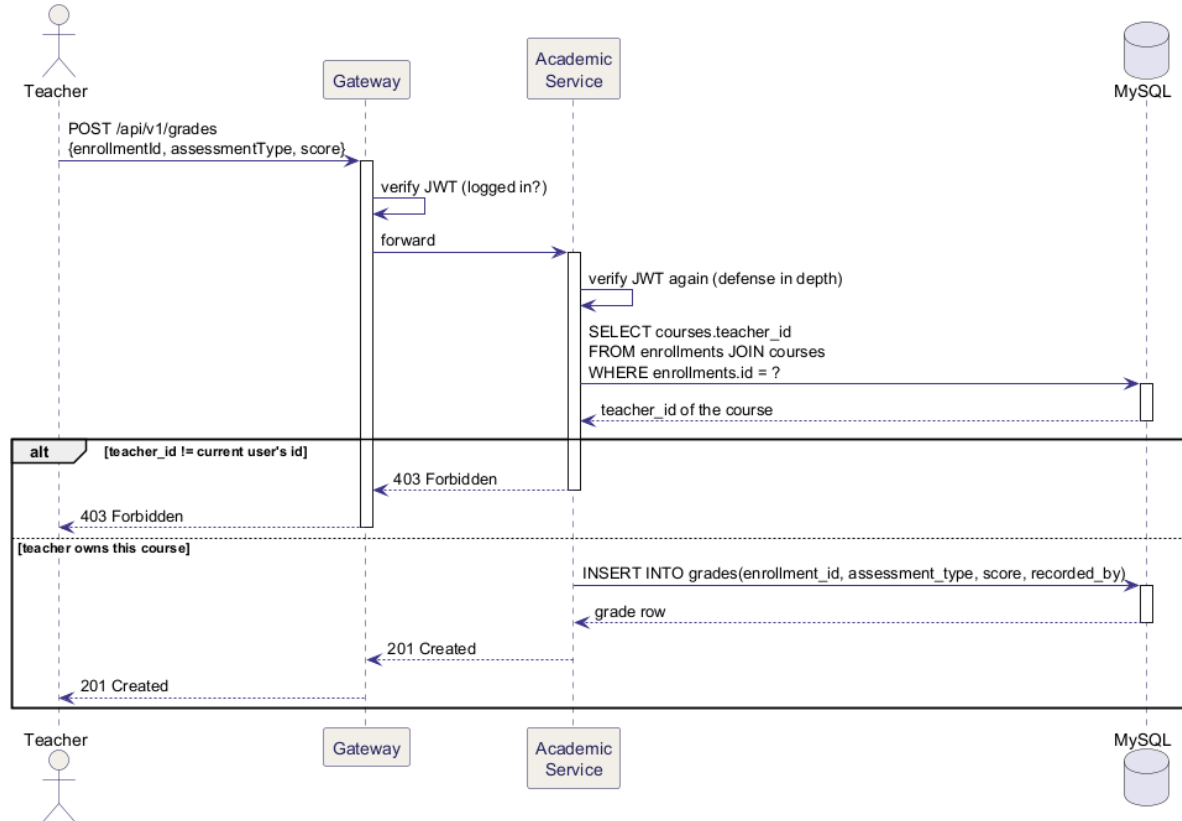


Figure 5.3: Sequence diagram: recording a grade, with teacher-ownership check and full activation bars (PlantUML source: `figures/smartschool/05_seq_grade.puml`)

5.4 State Transition Diagram: Invoice

Figure 5.4 shows the lifecycle of an `invoices.status` value using full UML state-machine notation: each state has an entry action, and each outgoing transition carries an explicit **guard condition** in square brackets alongside its triggering event. Payment collection (the `pending` → `paid` transition, guarded on `[gateway integrated]`) is planned; today an invoice can be created and, if applicable, cancelled, but there is no endpoint yet that performs the `paid` transition.

5.5 Activity Diagram: Enrollment-to-Invoice Workflow

Figure 5.5 lays out the same enrollment workflow as Figure 5.2, but as control flow rather than inter-participant messaging, organized into **swimlanes** – one per participant (Student, Academic Service, RabbitMQ, Finance Service) – so each action's owner is unambiguous at a glance. The fork explicitly rejoins at a single synchronization bar before the shared activity-final node, rather than each branch terminating on its own.

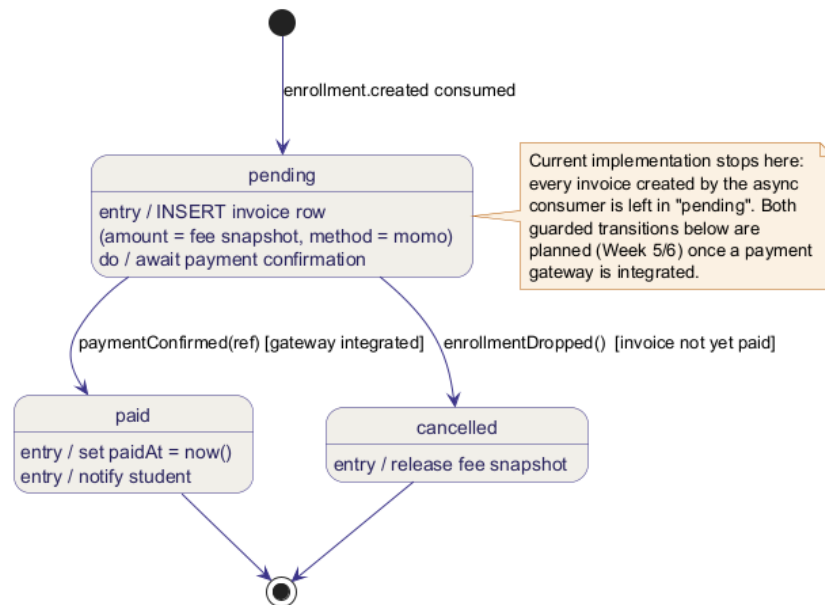


Figure 5.4: State transition diagram: invoice status lifecycle, with entry actions and guard conditions on every transition (PlantUML source: figures/smartschool/07_state_invoice.puml)

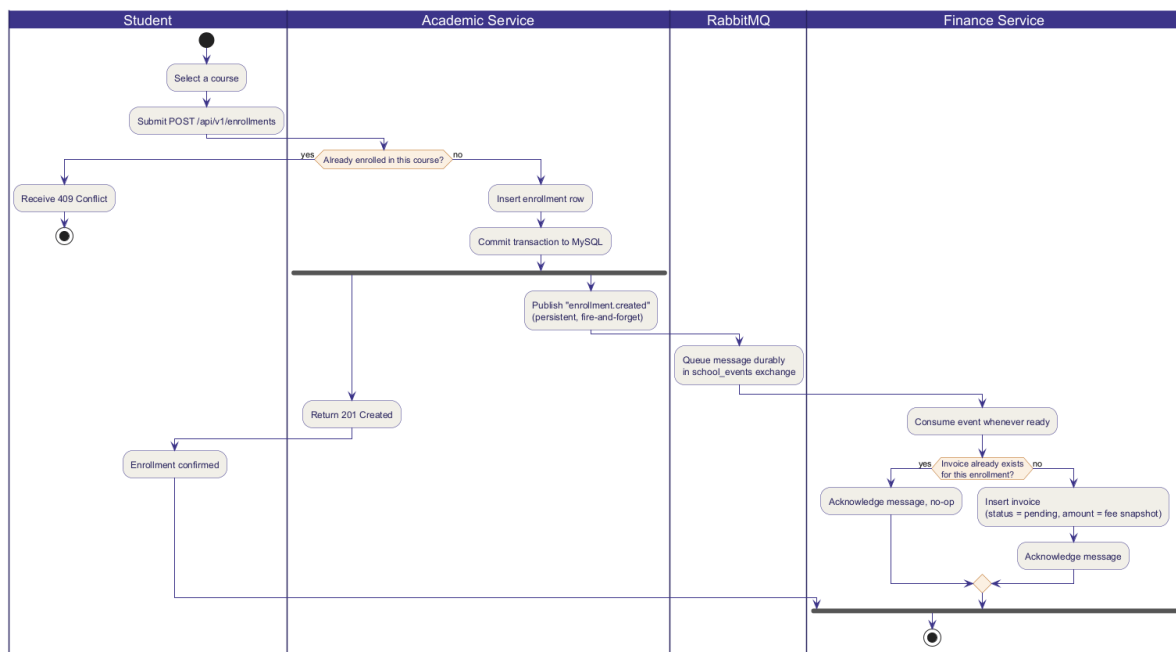


Figure 5.5: Activity diagram: enrollment-to-invoice workflow with swimlanes, a fork into an immediate HTTP response and an independent asynchronous branch, and a single join before the final node (PlantUML source: figures/smartschool/06_activity_enrollment.puml)

5.6 Object Diagram: Sample Enrollment Snapshot

Figure 5.6 is a concrete runtime snapshot – specific instances of Student, Teacher, Course, Enrollment, Grade, and Invoice, with real attribute values – as a worked instantiation of the class diagram in Figure 4.2. Every link is labelled with the same role name as the corresponding class-diagram relationship (*owns*, *has roster*, *contains*, *generates*, *teaches*, *records*), so the object diagram can be read as direct proof that the class diagram’s composition and aggregation relationships hold for real data.

It reuses the same example as the event contract in Chapter 6, so the two can be read side by side.

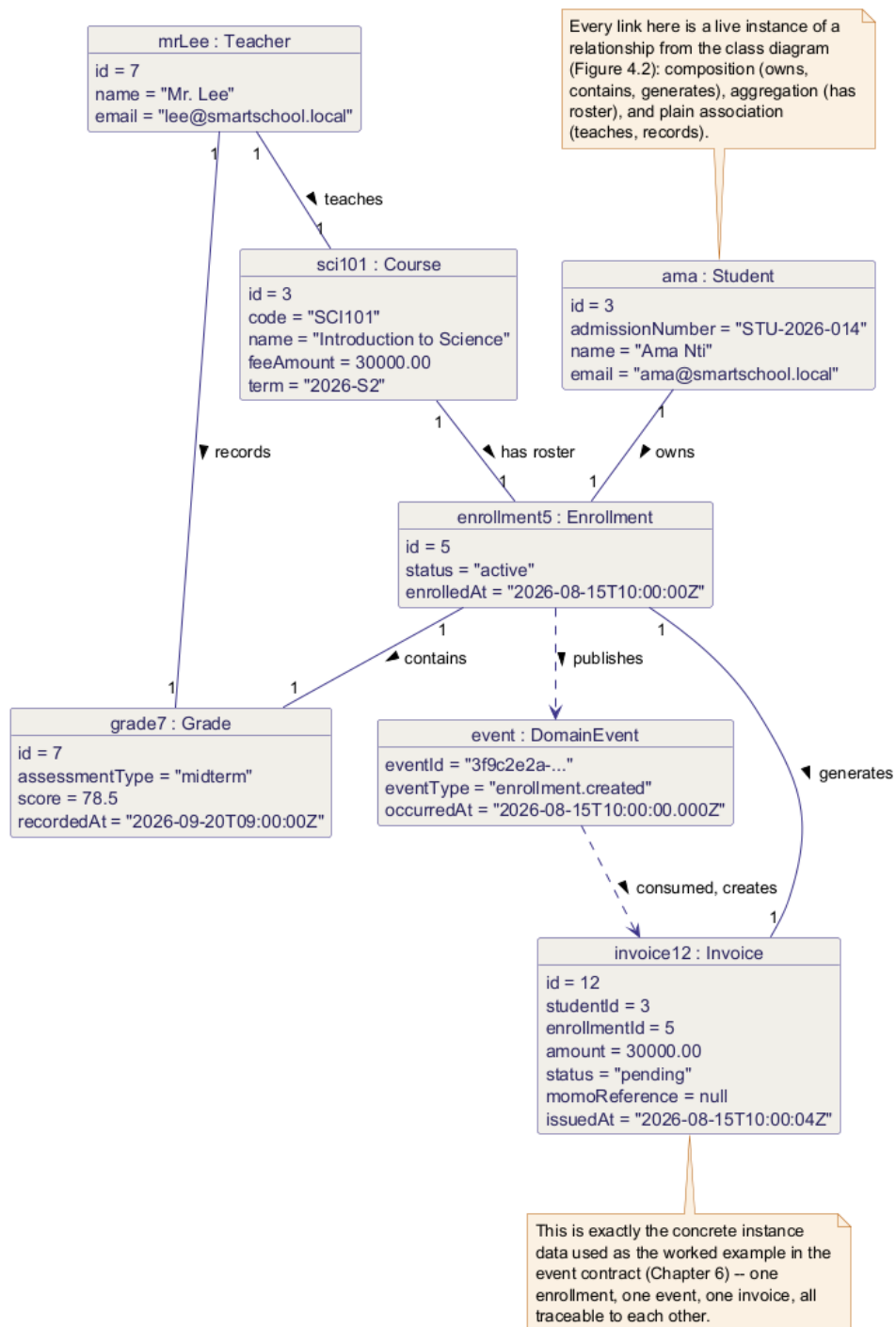


Figure 5.6: Object diagram: one enrollment, its triggering event, and the resulting invoice, with links naming the same roles as the class diagram (PlantUML source: figures/smartschool/08_object_snapshot.puml)

5.7 Idempotent Event Consumption

RabbitMQ guarantees *at-least-once* delivery, meaning the same `enrollment.created` message can, in principle, be delivered more than once (e.g. after a consumer restart before an ack is sent). The Finance Service handles this by attempting the invoice insert and treating a unique-constraint violation on `invoices.enrollment_id` as a successful no-op rather than an error – the second delivery

is acknowledged and discarded, and the invariant “one invoice per enrollment” never breaks. This is the deliberate mechanism, not an accident: idempotency is pushed down to a database constraint rather than implemented as fragile application-level deduplication logic.

Chapter 6

API Design

6.1 Routing Table (Gateway)

External path	Forwarded to	Auth
POST /api/v1/auth/register	auth-service /api/auth/register	None
POST /api/v1/auth/login	auth-service /api/auth/login	None
GET /api/v1/auth/me	auth-service /api/auth/me	JWT
GET /api/v1/reports	auth-service /api/reports	JWT (admin/teacher)
GET, POST /api/v1/courses	academic-service /api/courses	JWT (POST: admin)
GET, POST /api/v1/enrollments	academic-service /api/enrollments	JWT (POST: admin/student)
GET, POST /api/v1/grades	academic-service /api/grades	JWT (POST: admin/teacher)
GET /api/v1/finance/invoices	finance-service /api/invoices	JWT

6.2 Event Contract: enrollment.created

Published to the direct exchange `school_events` with routing key `enrollment.created`, persistent, and consumed by the `invoice_creation_queue`. Figure 5.6 in Chapter 5 renders this exact payload as an object diagram alongside the Student, Course, Enrollment, and Invoice instances it connects.

```
{
  "eventId": "3f9c2e2a-...",
  "eventType": "enrollment.created",
  "occurredAt": "2026-08-15T10:00:00.000Z",
  "data": {
    "enrollmentId": 5,
    "studentId": 3,
    "studentName": "Ama Nti",
    "studentEmail": "ama@smartschool.local",
    "courseId": 3,
    "courseCode": "SCI101",
    "courseName": "Introduction to Science",
    "feeAmount": "30000.00",
    "term": "2026-S2"
  }
}
```

Why `studentId` **is** `students.id`, **not** `users.id`. The event carries the domain-level student id directly, so Finance Service can insert straight into `invoices.student_id` with no extra lookup back to Academic Service – keeping the two services decoupled even at the data-access level.

Chapter 7

Security Design

7.1 Authentication and Session Model

Passwords are hashed with bcrypt (10 salt rounds) before storage; the raw password is never stored or logged. On successful login, the Auth Service issues a JWT (1-hour expiry) signed with a secret shared, via environment variable, across every service and the Gateway. This makes sessions stateless: any service can verify a token's authenticity without a network call back to Auth Service.

7.2 Defense in Depth

Two independent layers check every protected request:

1. **Gateway layer** – verifies the JWT's signature and expiry before proxying at all. An invalid/missing token is rejected with 401 and never reaches a backend service.
2. **Service layer** – the owning service (Auth, Academic, or Finance) re-verifies the JWT and additionally enforces role/ownership rules specific to its own resources (e.g. a Teacher may only grade courses they teach).

This split is deliberate: the Gateway is generic infrastructure answering "is this a logged-in user at all"; business-specific authorization rules stay with the service that owns the data.

7.3 OWASP Top 10 Mitigations

Risk	Mitigation
Injection	Every query in every service uses mysql2 parameterized statements (<code>pool.query(sql, [params])</code>); no request value is ever concatenated into a SQL string.
Broken Authentication	bcrypt password hashing, 1-hour JWT expiry, and the Gateway rate limiter (10 req/60s/IP) doubling as brute-force login throttling on <code>/api/v1/auth/login</code> . Accepted gap: password policy is length ≥ 8 only, no complexity rule – a deliberate scope limit for a course project.
Sensitive Data Exposure	<code>password_hash</code> is destructured out of every user object before it is returned (<code>toPublicUser()</code>), used by both <code>/register</code> and <code>/login</code> . Accepted gap: CORS is currently open on every service; judged low-severity since this is a Bearer-token API with no ambient cookie credential a cross-origin page could silently attach.

Chapter 8

Performance and Reliability

8.1 Rate Limiting

The Gateway applies `express-rate-limit` to every route except `/health`: 10 requests per 60 seconds per IP by default, both externally configurable (`RATE_LIMIT_MAX`, `RATE_LIMIT_WINDOW_MS`) without a code change. The 11th request in a window receives 429 with standard `RateLimit-*` headers showing remaining quota.

8.2 Load Testing

A k6 script (`load-testing/k6/gateway-load-test.js`) ramps to 20 virtual users against the Gateway. Two profiles are used:

- **Safety profile** (default limits) – proves the rate limiter engages under realistic concurrent load; the `rate_limited_responses` counter is non-zero.
- **Speed profile** (`RATE_LIMIT_MAX=100000`) – measures genuine backend latency once the limiter is not a factor; the script's threshold requires $p(95) < 500\text{ms}$.

8.3 Fault Tolerance in the Async Workflow

Both the publisher (Academic Service) and the consumer (Finance Service) connect to RabbitMQ with `retry-and-backoff` on startup, since a healthy container does not guarantee AMQP is immediately ready. Publishing is wrapped so that a RabbitMQ outage never fails or delays the enrollment HTTP response (Section 5.2); the message, once published, is durable and survives a broker restart. If Finance Service is stopped entirely, an enrollment still returns 201 immediately – the message queues up and is drained once the service restarts, with no resend needed from Academic Service.

Chapter 9

Deployment View

9.1 Container Topology

Container	Published port	Internal address	Depends on
db (MySQL 8)	3307 → 3306 (host-only, avoids clashing with a local install)	db:3306	—
rabbitmq	5673 → 5672, 15672 (mgmt UI)	rabbitmq:5672	—
auth-service	not published	auth-service:3000	db
academic-service	not published	academic-service:4000	db, rabbitmq
finance-service	not published	finance-service:4001	db, rabbitmq
gateway	8081 → 8080 (only published service)	gateway:8080	auth, academic, finance

The whole backend starts from the repository root with a single command, `docker compose up --build`; the frontend is a separate repository with its own `docker-compose.yml`, started the same way, and talks to the Gateway at `http://localhost:8081` directly from the browser.

Chapter 10

Planned Work (Weeks 5–6)

Not yet implemented, tracked here rather than silently absent.

- **Module feature (Week 5)** – one complete end-to-end feature for the group's assigned module: leading candidates are payroll computation (HR: `payroll_runs/payslips` already modeled) or invoice payment/reconciliation (Finance).
- **Automated testing and CI/CD (Week 5)** – a first batch of automated tests for the chosen feature, and a GitHub Actions pipeline that runs them on every push.
- **Full integration & documentation (Week 6)** – a single `docker-compose` command bringing up every service, the database, the Gateway, and the message broker together (already true for the backend today; needs re-verification on a clean machine), plus the completed UML diagram set (this SDD) and this SRS kept in sync with the final system.